
Dueling Network Architectures for Deep Reinforcement Learning

Ziyu Wang

Tom Schaul

Matteo Hessel

Hado van Hasselt

Marc Lanctot

Nando de Freitas

Google DeepMind, London, UK

ZIYU@GOOGLE.COM

SCHAUL@GOOGLE.COM

MTTHSS@GOOGLE.COM

HADO@GOOGLE.COM

LANCTOT@GOOGLE.COM

NANDODEFREITAS@GMAIL.COM

The Dueling Network

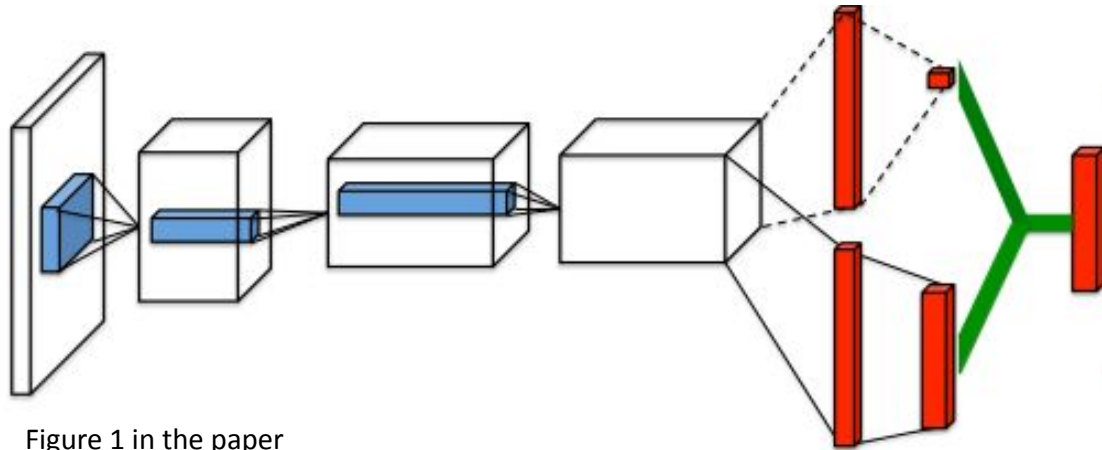


Figure 1 in the paper

- Learn separate representations of the state-value and state-action advantages
- Combine the value and advantage streams to produce action-value estimates for all actions of the input state

(Action) Advantage Function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

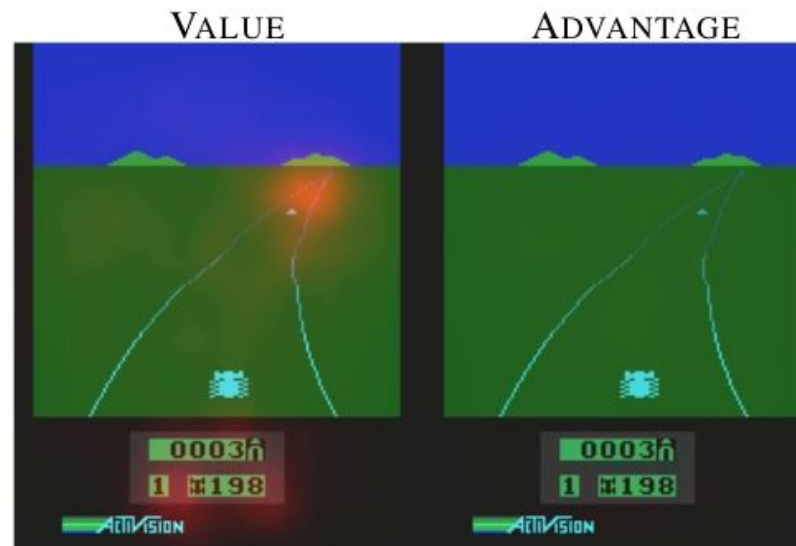
- What's the advantage of the optimal action for a state?

Motivation

- The choice of action may be inconsequential in many states
- The value of a state is important for bootstrapping (update value stream specific weights more often)
 - Q: But we used Q-values to bootstrap. How does value of a state help?

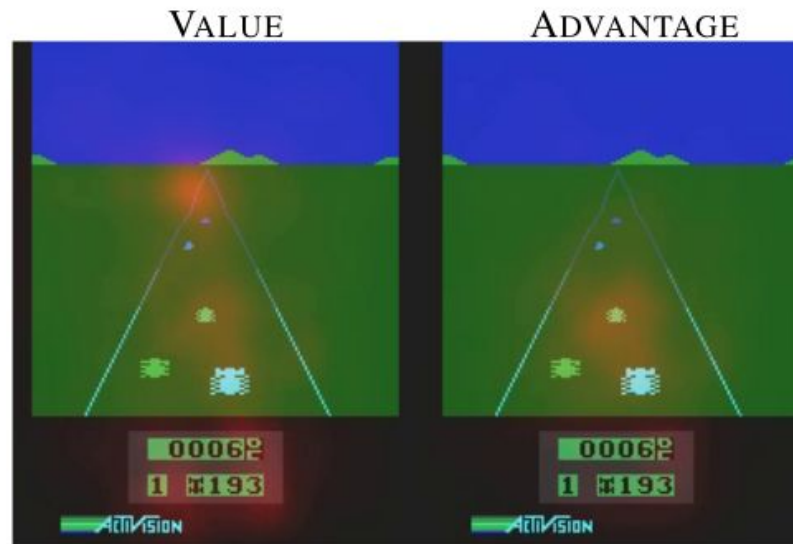
Motivation

- The choice of action may be inconsequential in many states
- Saliency Maps (Jacobian of the streams wrt the input)

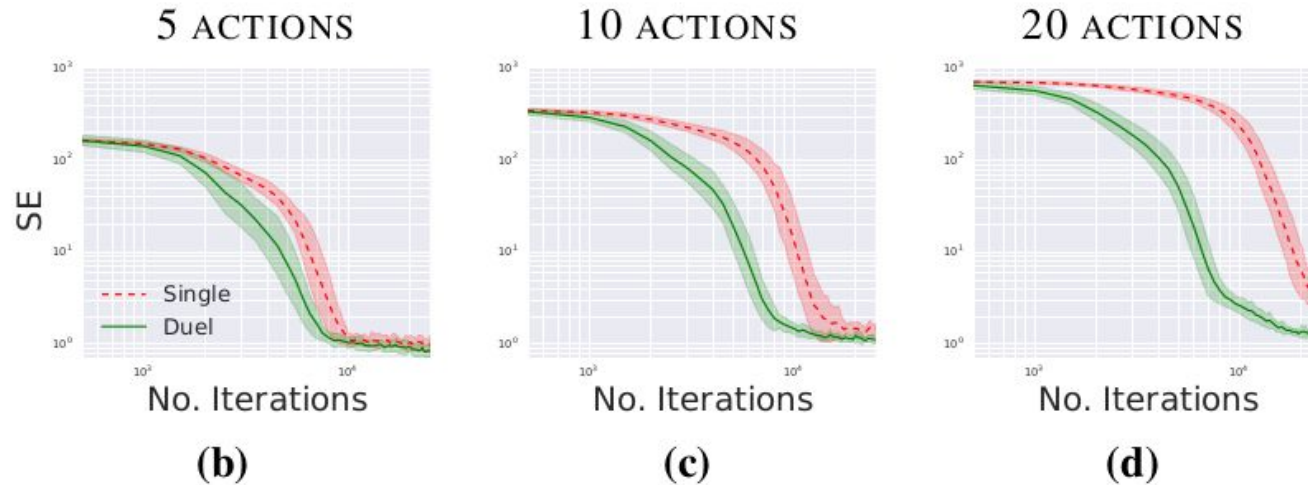


Motivation

- The choice of action may be inconsequential in many states
- Saliency Maps (Jacobian of the streams wrt the input)



Example Gains From Dueling



- Expected SARSA updates with epsilon-greedy behavior policy
- SE (squared estimation error of action values)

What Explains The Better Performance?

- Value of a state is updated more frequently as it is updated every time a state-action pair with the state is observed
 - Even more beneficial with larger number of actions

Combining The Value and Advantage Functions

$$Q(s, a; \theta, \alpha, \beta) = A(s, a; \theta, \alpha) + V(s; \theta, \beta)$$

- However, how do we ensure that the streams learn their intended functions?
 - Many combinations of the two terms on the right can give the same Q on the left

Combining The Value and Advantage Functions

- Value is set to action value of greedy action to force a correct correspondence between V and Q

$$Q(s, a; \theta, \alpha, \beta) = A(s, a; \theta, \alpha) - \max_a A(s, a; \theta, \alpha) + V(s; \theta, \beta)$$

Combining The Value and Advantage Functions

- Value is set to action value of greedy action to force a correct correspondence between V and Q

$$Q(s, a; \theta, \alpha, \beta) = A(s, a; \theta, \alpha) - \max_a A(s, a; \theta, \alpha) + V(s; \theta, \beta)$$

Alternately:

$$Q(s, a; \theta, \alpha, \beta) = \left(A(s, a; \theta, \alpha) - \frac{1}{|A|} \sum_a A(s, a; \theta, \alpha) \right) + V(s; \theta, \beta)$$

Loss Function and Parameter Updates

- Same as DDQN

ELIGIBILITY TRACES

Recall the n -step return target:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{\pi}(S_{t+n})$$

Consider targets that are weighted sums of n -step returns for different n .

Ex: $0.25 G_{t:t+2} + 0.75 G_{t:t+5}$

Generalize to λ -return:

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}, \lambda \in [0, 1]$$

What do the weights in G_t^{λ} sum to?

What is G_t^0 ? TD(0) that in 1-step TD

What is G_t^1 ? $(1-\lambda) [G_{t:t+1} + \lambda G_{t:t+2} + \lambda^2 G_{t:t+3} + \dots]$

$\downarrow$
MC

$(1-\lambda) [R_{t+1} + \gamma V_{\pi}(S_{t+1}) + \lambda (R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{\pi}(S_{t+2})) + \lambda^2 (\dots)]$

λ -return for an episodic task:

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n} + (?) G_{t:T}$$

What is the half-life time step T_{λ} ? The step at which the weight is $1/2$ of the initial?

Offline λ -return algorithm.

$$w_{t+1} = w_t + \alpha (G_t^\lambda - \bar{v}(s_t, w_t)) \nabla \bar{v}(s_t, w_t)$$

Online or Offline?

G_t^λ requires taking the forward view.
We must wait for the future states to present themselves.

TD(1) approximates the above using a backward view.

→ Update weights in an online manner

→ Use Eligibility Traces to do the same.

Eligibility trace vector z_t :

$$z_1 = \bar{0}$$

$$z_t = \gamma \lambda z_{t-1} + \nabla \hat{V}(s_t, w_t)$$

$$0 \leq t \leq T.$$

[A short-term memory of which elements of the weight vector have contributed]

What is z_t when $\hat{V}$ is a linear approximation?

Consider:

Semi-gradient TD(1) for prediction

For each step in an episode:

$$z = \gamma \lambda z + \nabla \hat{V}(s, w)$$

$$\delta = R + \hat{V}(s', w) - \hat{V}(s, w)$$

$$w = w + \alpha \delta z$$

As λ gets larger, more of the previously visited states' values are updated on update of w .

TD(0)??

TD(1) ~

TD(1): As λ increases, a longer sequence of past states is given credit for the TD error. Equivalently, given a length of sequence of past states, increasing λ gives more credit to states in the past for the error.

AVERAGE REWARD

→ Continuing Task setting without discounting

Average reward $r(\pi)$ when following policy π ,

$$r(\pi) \triangleq \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=1}^h E[R_t | S_0, A_{0:t-1} \sim \pi] \quad \left\{ \begin{array}{l} \text{Limiting} \\ \text{time-average} \\ \text{of expected rewards.} \end{array} \right.$$

$$= \lim_{t \rightarrow \infty} E[R_t | S_0, A_{0:t-1} \sim \pi]$$

(Assuming reward distribution converges in the limit as $t \rightarrow \infty$)

Further assume that in the limit as $t \rightarrow \infty$, the steady state probability distribution exists and is $\mu_\pi(s)$. Also assume it is independent of initial state S_0 .

$$r(\pi) = \lim_{t \rightarrow \infty} E[R_t]$$

$$= \sum_s \mu_\pi(s) \sum_a \sum_{s', r} r \cdot p(s', r | s, a) \pi(a | s)$$

$$= \sum_s \mu_\pi(s) \sum_a r \cdot p(r | s, a) \pi(a | s)$$

Write $\mu_\pi(s')$ in terms of $\mu_\pi(s)$.

AVERAGE REWARD

The goal is to find π that maximizes $r(\pi)$.

let $v_{\pi}^V(s)$ be the value of state s , given a discount factor V and policy π .

Assume steady state distribution $\mu_{\pi}(s)$ exists and is independent of the initial state.

Consider $J(\pi) = \sum_s \mu_{\pi}(s) v_{\pi}^V(s)$

Show that $J(\pi) = r(\pi)/(1-V)$

That is ordering of $r(\pi)$ with respect to policies π is the same as that of $J(\pi)$.

AVERAGE REWARD

The Bellman Equations:

We need differential returns & differential value function.

$$G_t \triangleq R_{t+1} - \gamma(\pi) + R_{t+2} - \gamma(\pi) + \dots$$

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

$$q_{\pi}(s, a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

To obtain the Bellman Equations, replace rewards in the equations for the case of discounted reward by differential reward.

$$\begin{aligned} V_{\pi}(s) &= E_{\pi}[R_{t+1} - \gamma(\pi) + V_{\pi}(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} (\gamma - \gamma(\pi) + V_{\pi}(s')) P(s', a | s, a) \end{aligned}$$

$$V_{\pi}(s) =$$

$$q_{\pi}(s, a) =$$

$$q_{\pi}(s, a) =$$

AVERAGE REWARD

TD error:

$$\delta_t = R_{t+1} - \bar{R}_{t+1} + \hat{V}_t(S_{t+1}, w_t) - \hat{V}_t(S_t, w_t)$$

For action-values and SARSA:

$$\delta_t = R_{t+1} - \bar{R}_{t+1} + \hat{Q}_t(S_t, A_{t+1}, w_t) - \hat{Q}_t(S_t, A_t, w_t)$$

Semi-gradient SARSA update:

$$w_{t+1} = w_t + \alpha \delta_t \nabla \hat{Q}_t(S_t, A_t, w_t)$$

Differential Semi-Gradient SARSA

Initialize S, A ; $\bar{R} = 0$.

At each time step:

Execute action A . Observe S' , R .

Choose $A' \sim \epsilon$ -greedy over values $\hat{Q}_t(S', a, w)$, $\forall a$.

$$\delta = R - \bar{R} + \hat{Q}_t(S', A', w) - \hat{Q}_t(S, A, w)$$

$$\bar{R} = \bar{R} + \beta \delta$$

[This makes sense as δ is a sample estimate of average reward $r(\pi)$.]
(Show Rn above)

$$w = w + \alpha \delta \nabla \hat{Q}_t(S, A, w)$$

$$S = S', A = A'$$